

UiB-AlgoRythm: A Robust Anytime Heuristic Solver for Rooted Maximum Agreement Forest

Juni Weisteen Bjerde¹, Wim Van den Broeck¹, Krishnan Dehaleesan¹, Yash Hiren More¹, Jakob Rødal Skaar¹, Tomáš Turek¹, Sam Urmian²

¹ Department of Informatics, University of Bergen, Norway

² Department of Informatics and Centre for the Science of Learning & Technology (SLATE), University of Bergen, Norway

juni.bjerde@uib.no, wim.broeck@uib.no, krishnan.dehaleesan@uib.no, yash.h.more@uib.no,

jakob.r.skaar@student.uib.no, tomas.turek@uib.no, sam.urmian@uib.no (contact)

Abstract

We describe **UiB-AlgoRythm**, our submission to the PACE 2026 Heuristic Track. The problem is rooted *Maximum Agreement Forest* (MAF) on two rooted binary phylogenetic trees T_1, T_2 over a common leaf set of size n : partition the leaves into the fewest agreement components, each inducing the same rooted topology in both trees with the components edge-disjoint in each tree, minimizing the component count k (equivalently the rooted SPR distance $k - 1$). Our solver is a single C++17 file, is anytime, uses at most 8 GB, and is SIGTERM-safe. The design is dominated by one overriding rule of the track: *a single infeasible or empty output disqualifies the entire submission*. Feasibility and robustness therefore take precedence over solution quality everywhere. We combine a lossless Bordewich–Semple subtree kernelization, a merge-based constructive heuristic with a stack of improvement passes, a never-regress exact Whidden–Beiko–Zeh branch-and-bound repair on small clean common clades, and an anytime restart portfolio, all behind an always-on feasibility gate and an async-signal-safe best-so-far flush.

2012 ACM Subject Classification Theory of computation → Fixed parameter tractability; Applied computing → Molecular evolution

Keywords and phrases Maximum agreement forest, phylogenetics, rooted SPR distance, heuristic, kernelization, anytime algorithm, PACE 2026

Category PACE 2026 Heuristic Track solver description

1 Problem and design constraints

An *agreement forest* of T_1 and T_2 is a partition of the leaf set into components such that (i) each component induces the same rooted topology in both trees (children unordered, degree- ≤ 1 vertices suppressed) and (ii) the components’ minimal spanning subtrees are edge-disjoint in each tree (node-disjoint, for binary trees). The objective is to minimize k . The input is a PACE `.nw` instance (`#p 2 n`, then two Newick trees on labels $1, \dots, n$); the output is one Newick component per line, `;-`terminated, singletons as bare integers.

The track sends **SIGTERM** at the five-minute mark, then **SIGKILL**; a *correct* forest must have been printed by then. Two failure modes are fatal—an infeasible forest and an *empty* output—the latter typically caused by an uncatchable **SIGSEGV** from stack overflow on a deeply nested (caterpillar/ladder) input. Our engineering targets never triggering either mode.

2 Algorithm pipeline

Algorithm 1 is the top-level anytime driver; the paragraphs below elaborate each step.

■ **Algorithm 1** Anytime rooted-MAF heuristic (single worker thread, 1 GiB stack).
 PUBLISHIFBETTER(F) installs F as the new best iff F is feasible and $k(F)$ strictly improves.

```

Input :rooted binary trees  $T_1, T_2$  on leaves  $1, \dots, n$ 
publish(singletons) // valid  $k=n$ ; cached and flushed on SIGTERM
 $(T_1, T_2), \pi \leftarrow \text{kernelize}(T_1, T_2)$  // Bordewich-Semple; adopt if  $\geq 10\%$  smaller
 $F \leftarrow \text{smaller of } \text{cladeSweep}(T_1), \text{cladeSweep}(T_2)$ 
44  $F \leftarrow \text{improveToFixpoint}(F)$ ; publishIfBetter( $F$ ) // fixpoint of local moves
 $F \leftarrow \text{exactCorePolish}(F)$ ; publishIfBetter( $F$ ) // never-regress
foreach seed  $S$  in portfolio do
  | publishIfBetter(improveToFixpoint( $S$ ))
while not expired and  $k(F) > 1$  do
  | publishIfBetter(improveToFixpoint(shatter(best))) // LNS restart
return best // expanded through kernel map  $\pi$  at output

```

45 Singleton fallback published first.

46 The moment n is read from the header—before any search—the all-singleton forest ($k = n$,
 47 always feasible) is cached, so a SIGTERM at any later instant flushes a valid forest rather than
 48 nothing.

49 Lossless common-subtree kernelization.

50 We iteratively collapse every *common cherry* (two leaves that are siblings in both trees)
 51 into a single super-leaf, cascading to collapse maximal common pendant subtrees. This is
 52 the classic Bordewich–Semple subtree reduction [1]: a common pendant subtree is intra-
 53 component in every MAF, so the reduction is distance/optimality preserving and any valid
 54 forest on the kernel expands to a valid forest on the original instance. We solve the smaller
 55 kernel and expand each super-leaf back to its subtree at output time. The kernel is adopted
 56 only on a meaningful ($\geq 10\%$) reduction; a barely-reduced kernel of a deep common ladder
 57 is discarded, since solving the full instance is then both faster and better.

58 Construction: common-clade sweep.

59 The initial forest is built by a *common-clade sweep*. Scanning one tree’s clades in decreasing
 60 size, we keep every clade that also occurs in the other tree and is edge-disjoint from what is
 61 already kept, so the shared backbone becomes a few large components and only the genuinely
 62 conflicting leaves remain as singletons; run from each tree’s viewpoint, we keep the smaller
 63 forest.

64 Improvement: a fixpoint of complementary local moves.

65 The forest is then reduced by a stack of local moves, iterated to a fixpoint: singleton
 66 pairing and packing; windowed *component packing* and *corridor merges* that re-group the
 67 components over a small region of one tree; greedy *merges* of two components whose union
 68 is still a common agreement subtree; and a bounded exact-cover *window repair* that re-
 69 optimizes a small window to its local optimum. On large instances a *conflict-directed* repair
 70 unblocks a merge by splitting the blocking component when that still lowers the count.
 71 Every move tests edge-disjointness against the live forest before it is applied, so feasibility
 72 is an invariant of the entire stack.

Never-regress exact-core repair.

As a final polish on the converged forest, small *clean* common clades—clades appearing in both trees that are exactly a union of whole current components—are re-solved to proven optimum by an exact rooted-MAF branch-and-bound in the style of Whidden, Beiko, and Zeh [2]. The exact core is node- and wall-time-capped; if it does not finish, the heuristic result is kept, and a solved core is spliced back only when it *strictly* reduces the component count, so this step can never worsen the forest. Running it early was found to steal budget from the more productive search phases, so it is deferred to the end.

Seed portfolio and anytime restart.

Because the greedy stack is basin-dependent, we restart it from a portfolio of structurally different initial forests: a maximum agreement subtree packing, greedy and greedy-partition constructors, a cherry-cut beam, exact common-cluster decompositions, and a pair portfolio. Each is fed through the same improvement stack and adopted only if it beats the incumbent. The remaining budget drives a large-neighborhood restart loop: it shatters a random subset of the best forest’s components back into singletons (alternating gentle and aggressive ratios) and lets the stack re-glue that region a different way, always keeping the best valid forest seen.

Always-on feasibility gate.

Every candidate forest is re-checked before it may become the published best: the labels must partition $1, \dots, n$, each component must induce the same rooted topology in both trees, and the components must be edge-disjoint in both trees. An infeasible candidate is refused and the previous valid best is retained. The check is near-linear, comparing oversized near-identical components by a 128-bit canonical topology hash rather than the quadratic string form that would blow up on deep shapes.

3 Robustness engineering

The distinctive part of the submission is defensive: correctness must survive adversarial input shapes and an asynchronous kill.

Deep-input stack safety.

Every recursive routine (Newick parsing and serialization, restricted-topology construction, kernel expansion) recurses to a depth that is linear in the tree *depth*, which for a caterpillar is n . On the default ≈ 8 MB stack this overflows and **SIGSEGVs**—silently, with empty output—at n of the order of 10^5 . We run the entire pipeline (parse, kernelize, solve, flush) on a single worker thread whose 1 GiB stack is set on the thread attribute, independently of the **ulimit** soft limit. Only touched pages are resident, costing no real memory on the shallow common case. The process stays single-threaded in the PACE sense: the main thread only blocks joining the worker.

Async-signal-safe termination.

The best-so-far forest is kept as a single pre-serialized, newline-terminated string in a double-buffered cache. The **SIGTERM**/**SIGINT** handler performs no allocation and no locking: it reads the current buffer pointer and length, writes them to file descriptor 1 with a single **write**,

and exits. On normal exit we first ignore SIGTERM and emit through the same write path, so a late signal can neither interleave with nor duplicate output.

4 Correctness and verification

The exact branch-and-bound is fuzz-verified against a brute-force oracle (2000+ instances, $n \leq 11$) and an independent reference implementation ($n \leq 15$) with zero mismatches; kernelization is optimality-preserving on thousands of small instances and its leaf expansion is a bijection. Beyond the always-on feasibility gate, every output is cross-checked by an independent dependency-free checker that shares no code with the solver. Re-running end to end for this submission: the official example is solved optimally; across 550 random instances (250 built to trigger kernelization, which fired on 62) every output was feasible and matched the brute-force optimum (0 infeasible, 0 suboptimal); and a deep caterpillar of $n=120,000$ under an 8 MB stack neither crashed nor timed out—identical trees collapse to the exact $k=1$ forest, and a mid-search SIGTERM flushes a valid full partition. The two disqualifying failure modes are thus both covered.

5 Interface and reproducibility

The solver is a single C++17 file with no external dependencies, built with g++ at `-O3 -std=c++17 -pthread`. It reads one instance on `stdin` and writes the forest to `stdout`; the budget is `PACE_TIME_LIMIT` seconds (default 298, a margin below the five-minute mark). Two toggles, `PACE_NOKERNEL` and `PACE_NOEXACT`, disable kernelization and exact-core repair for ablation. All algorithms are an original implementation; no third-party solver source was copied.

References

- 1 M. Bordewich and C. Semple. On the computational complexity of the rooted subtree prune and regraft distance. *Annals of Combinatorics*, 8(4):409–423, 2005.
- 2 C. Whidden, R. G. Beiko, and N. Zeh. Fixed-parameter algorithms for maximum agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013.